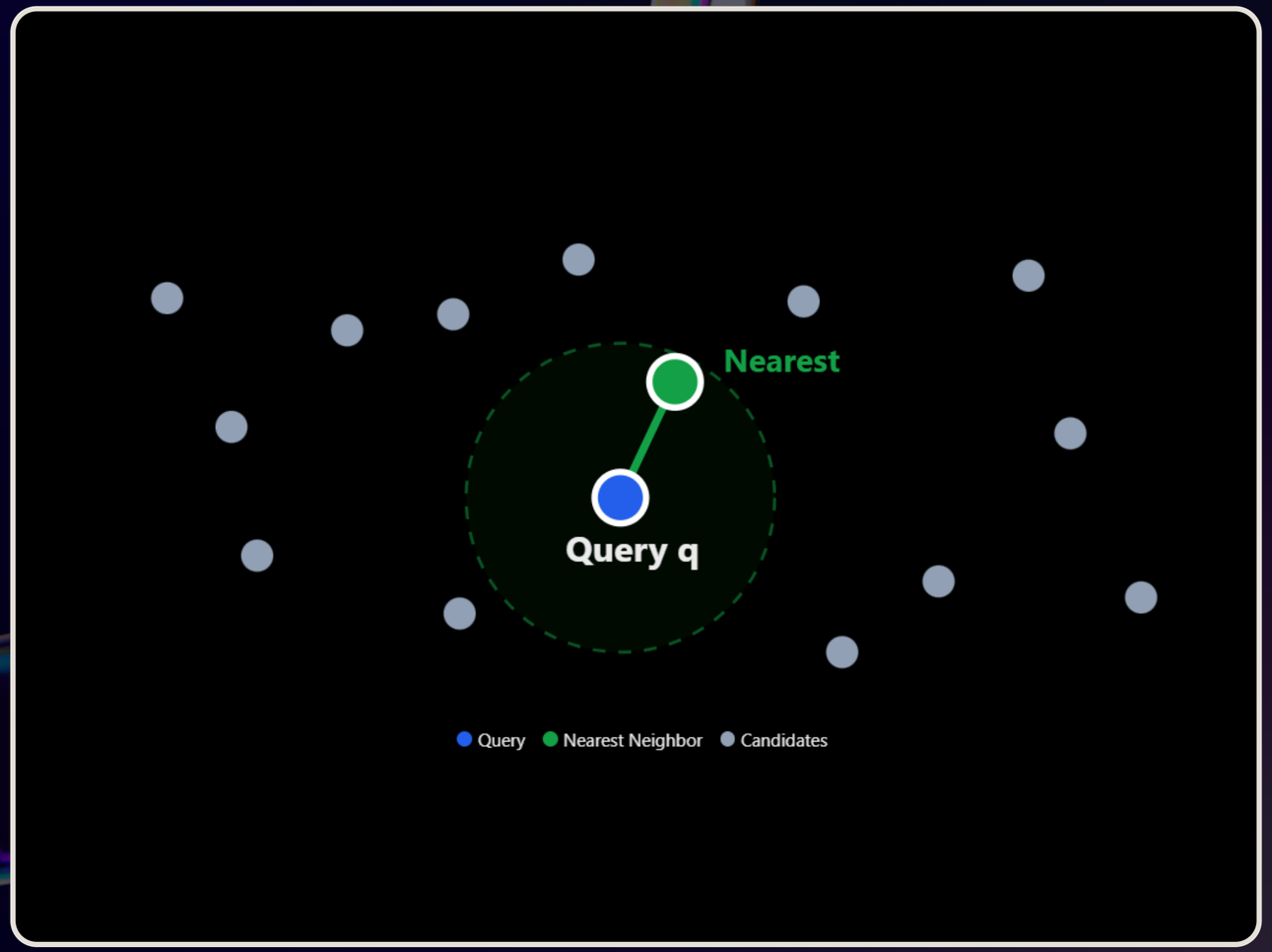
Project GPU-Accelerated Exact Nearest Neighbor Search

Using CUDA

Department of Computer Science

Presented by: Thanaphat 6710450929

Exact Nearest Neighbor Search



Problem:

ค้นหา Vector ที่ใกล้ Query มากที่สุดจาก Candidates จำนวน N ตัว

Exact Search:

ตรวจสอบทุก Candidate เพื่อให้ได้ผลลัพธ์ที่ตรงตามนิยาม ไม่ใช้ Approximation

Motivation:

การคำนวณ Distance ของแต่ละ Candidate เป็นอิสระต่อกัน จึงเหมาะกับ Data Parallelism

Challenge:

GPU มี Parallelism สูง แต่มีต้นทุน Memory Transfer และการจัดการข้อมูล

How Exact Nearest Neighbor Search Works

Squared Euclidean Distance ($L2^2$)

$$d_i = \sum_{j=0}^{D-1} (x_i[j] - q[j])^2$$

q = Query Vector

x_i = Candidate Vector ตัวที่ i

N = จำนวน Candidates

D = จำนวน Dimensions

d_i = ระยะทางยกกำลังสองระหว่าง Query กับ Candidate ตัวที่ i

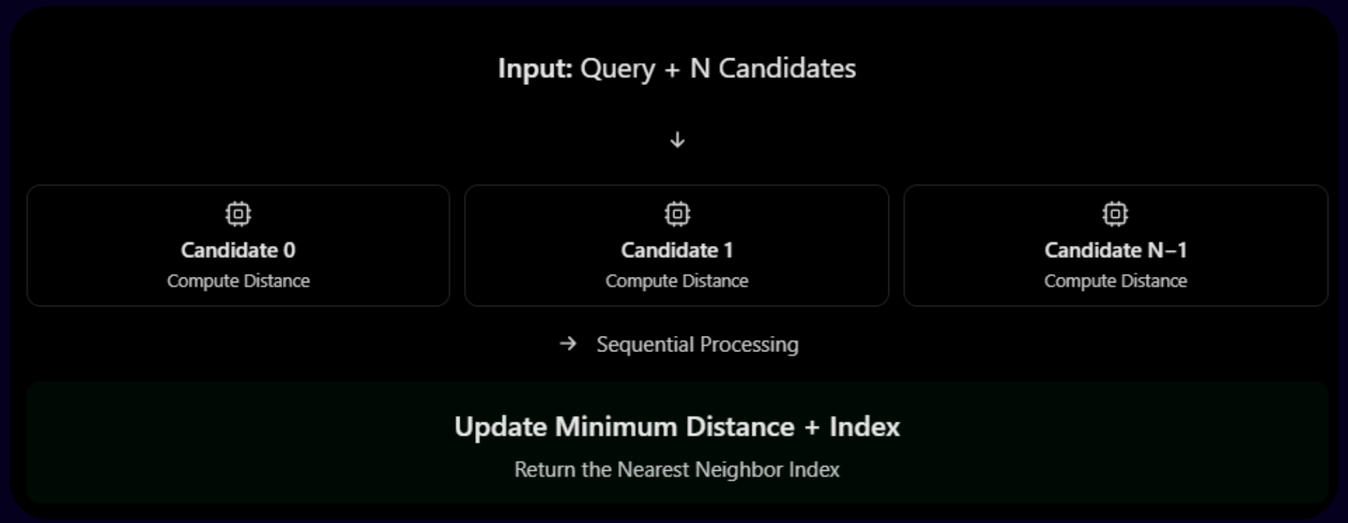
Input: Query + N Candidates

Compute Distance for Every Candidate

Find Minimum Distance

Output: Nearest Neighbor Index

CPU Baseline

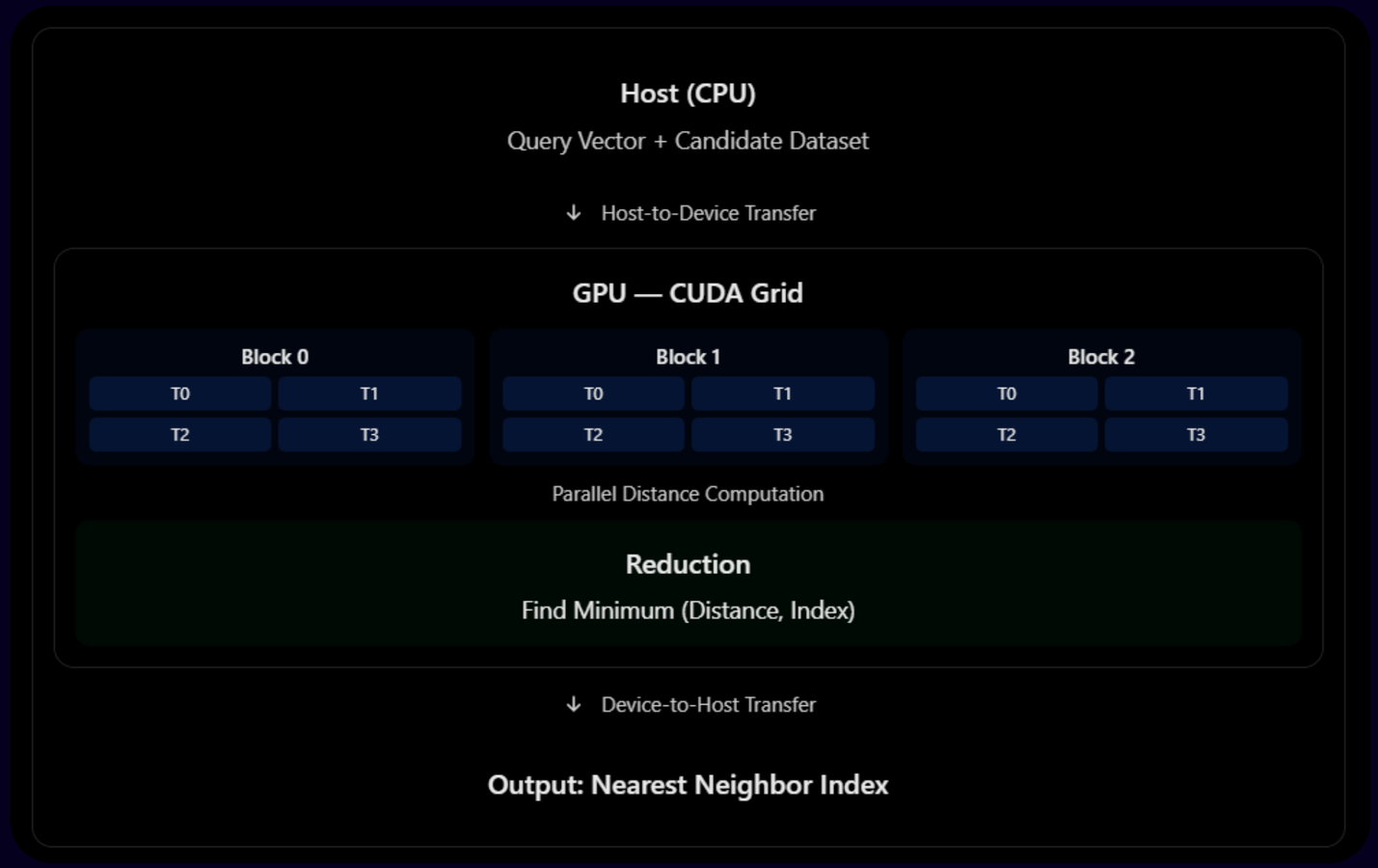


Purpose of CPU Baseline

- Correctness: ใช้ตรวจสอบความสอดคล้องของผลลัพธ์กับ CUDA V0–V5
- Performance: ใช้เป็น Baseline สำหรับเปรียบเทียบ GPU Raw End-to-End Latency

- **Exhaustive Search: ตรวจสอบ Candidates ทุกตัว**
- **Sequential Execution: คำนวณทีละ Candidate**
- **Time Complexity: O(ND)**
- **Precision: Float32**
- **Tie-breaking: เลือก Index น้อยที่สุดเมื่อระยะทางเท่ากัน**

CUDA Architecture



CUDA Optimization Evolution: V0–V3

V.0

Baseline CUDA

AoS + Parallel Distance + Serial Reduction

V.1

Memory Layout

AoS → SoA for coalesced memory access

V.2

Parallel Reduction

Replace serial minimum search with hierarchical shared-memory reduction

V.3

Warp-Level Reduction

Use Warp Shuffle to reduce shared-memory and synchronization overhead

Parallel Reduction Strategies

Serial Reduction (V0–V1)

- ใช้ Thread เดียวค้นหา Minimum Distance จาก Candidates ทั้งหมด
- Limitation: Sequential Bottleneck

Shared-Memory Reduction (V2)

- ใช้หลาย Threads คำนวณ Minimum ภายในแต่ละ Block
- ใช้ Shared Memory และ Hierarchical Reduction
- Benefit: เพิ่ม Parallelism และลด Serial Bottleneck

Warp-Shuffle Reduction (V3)

- ใช้ Warp Shuffle เพื่อแลกเปลี่ยนข้อมูลระหว่าง Threads ภายใน Warp
- ลดการใช้ Shared Memory และ Synchronization บางส่วน
- Benefit: ลด Communication Overhead ภายใน Warp

Complexity Comparison :

Strategy	Work	Span
Serial Reduction	O(N)	O(N)
Parallel Reduction Tree	O(N)	O(log N)

Experimental Setup — V0–V3

1. Hardware & Software

Component	Specification
GPU	NVIDIA GeForce RTX 3060 Laptop
Compute Capability	8.6
CPU	Intel Core i7-11370H
CUDA Toolkit	13.3
Operating System	Windows 11
Compiler	MSVC 19.51
Build Configuration	Release, Fast Math Disabled

2. Synthetic Workload Configuration

Parameter	Values
Dataset Type	**Synthetic**
Candidates (N)	1,024 / 16,384 / 262,144 / 1,048,576
Dimensions (D)	8 / 32 / 128
Total Workloads	12 Configurations
Data Precision	Float32
Query Type	Single Query

3. Benchmark Methodology

- Warm-up: 10 Runs
- Measurement: 100 Runs per Configuration
- Metric: Median Execution Time (ms)
- Correctness: ตรวจสอบผลลัพธ์ก่อนยอมรับผลการทดลอง
- Comparison: CPU Baseline vs CUDA V0–V3

Measurement Methodology

Objective: แยกเวลาประมวลผล CUDA Kernel ออกจากเวลาทำงานทั้งระบบ เพื่อประเมินว่า GPU เร็วกว่า CPU จริงหรือไม่ ภายใต้ขอบเขตการวัดที่เทียบกันได้

1. CPU Raw End-to-End (CPU-E2E-RAW)
- วัดเวลาตั้งแต่เรียก CPU Search จนได้รับผลลัพธ์ รวม Validation และ Distance Computation

2. GPU Kernel-Only (BND-KERNEL)
- วัดเฉพาะ CUDA Kernel Execution รวม Distance Computation และ Reduction ไม่รวม Allocation, Conversion และ Transfer

3. GPU Raw End-to-End (BND-E2E-RAW)
- วัดเวลาตั้งแต่เรียก GPU Search จนได้รับผลลัพธ์ รวม Allocation, Data Conversion (ตามเวอร์ชัน), Host↔Device Transfer, Kernel Execution และ Synchronization

4. GPU Prepared / Steady-State
- วัดการค้นหาเมื่อ Dataset และ Workspace ถูกเตรียมไว้สำหรับใช้ซ้ำ โดยแยกต้นทุน Preparation/Construction ออกจากเวลาค้นหา

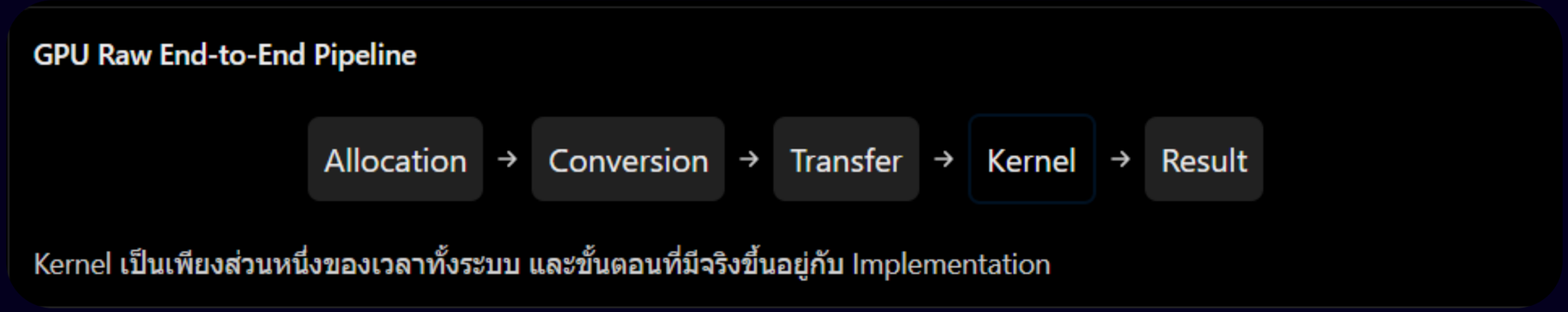
Performance Metric — Speedup

$$\text{Speedup} = \frac{T_{\text{CPU}}}{T_{\text{GPU}}}$$

Primary Comparison

CPU Raw End-to-End vs GPU Raw End-to-End

Kernel Speedup ≠ End-to-End Speedup



Kernel Optimization
V0–V3 optimized GPU distance computation and reduction.
Remaining Overhead
Memory Allocation, Data Conversion, and Host↔Device Transfer contribute to Raw E2E latency.
Key Finding
Faster CUDA kernels do not necessarily result in faster GPU End-to-End execution.

Next Step
V4: Optimize data preparation through Cache-Blocked Transpose.
V5: Improve workload parallelism and amortize overhead through Batch Query.

V0–V3 Kernel Optimization Results

Hypothesis Results

Hypothesis	Description	Result
H2	V1 Kernel faster than V0	REJECTED
H3	V2 Kernel faster than V1	SUPPORTED (12/12)
H4	V3 Kernel faster than V2	REJECTED

Key Performance Results

V2 vs V1 — Median Kernel Speedup

2.233×–84.286×

Speedup range across 12 workloads

V2 Faster than V1

12/12

Workloads supported

Performance Analysis

- V0 → V1: Memory Layout - AoS → SoA ช่วยปรับรูปแบบ Memory Access แต่ไม่ได้เพิ่ม Kernel Performance ทุก Workload
- V1 → V2: Hierarchical Reduction - ลด Serial Reduction Bottleneck และผ่านเกณฑ์ Kernel Performance 12/12 Workloads
- V2 → V3: Warp Shuffle - ลดการพึ่งพา Shared Memory แต่ไม่ได้เร็วกว่า V2 ทุก Workload

Single-Query Limitation — GPU vs CPU

Key Findings

- H1: REJECTED (12/12)
- GPU V3 Raw E2E was slower than CPU in all 12 single-query workloads.
- Kernel Speedup ≠ End-to-End Speedup
- แม้ Kernel ได้รับการ Optimize แล้ว แต่ GPU ยังไม่สามารถเอาชนะ CPU เมื่อวัดเวลาทั้งระบบ

Performance Bottlenecks

Bottleneck	Description
Memory Allocation	การจัดสรรหน่วยความจำ GPU เพิ่มภาระงานต่อการสืบค้น
AoS → SoA Transpose	การแปลงรูปแบบข้อมูลเพิ่มเวลาในการเตรียมการ
Host ↔ Device Transfer	การถ่ายโอนข้อมูลระหว่าง CPU และ GPU เพิ่มความหน่วง
Synchronization & Cleanup	การรอและการล้างทรัพยากรเพิ่มภาระงานในการดำเนินการ

Key Takeaway

Single Query could not amortize GPU overhead effectively.
Next: V4 optimizes data preparation, followed by V5 Batch Query to increase useful GPU work.

V4 — Cache-Blocked Transpose Optimization

Optimization Technique

Cache-Blocked Transpose

- Divide data into smaller tiles to improve cache locality.
- Optimize AoS → SoA memory access patterns.
- Tested tile sizes: 32×32, 64×32, 64×64.

V4 Experimental Results

Comparison	Supported	Rejected	Inconclusive
Host Transpose	18	18	0
GPU Raw E2E	18	8	10
CPU vs GPU Raw E2E	0	144	0

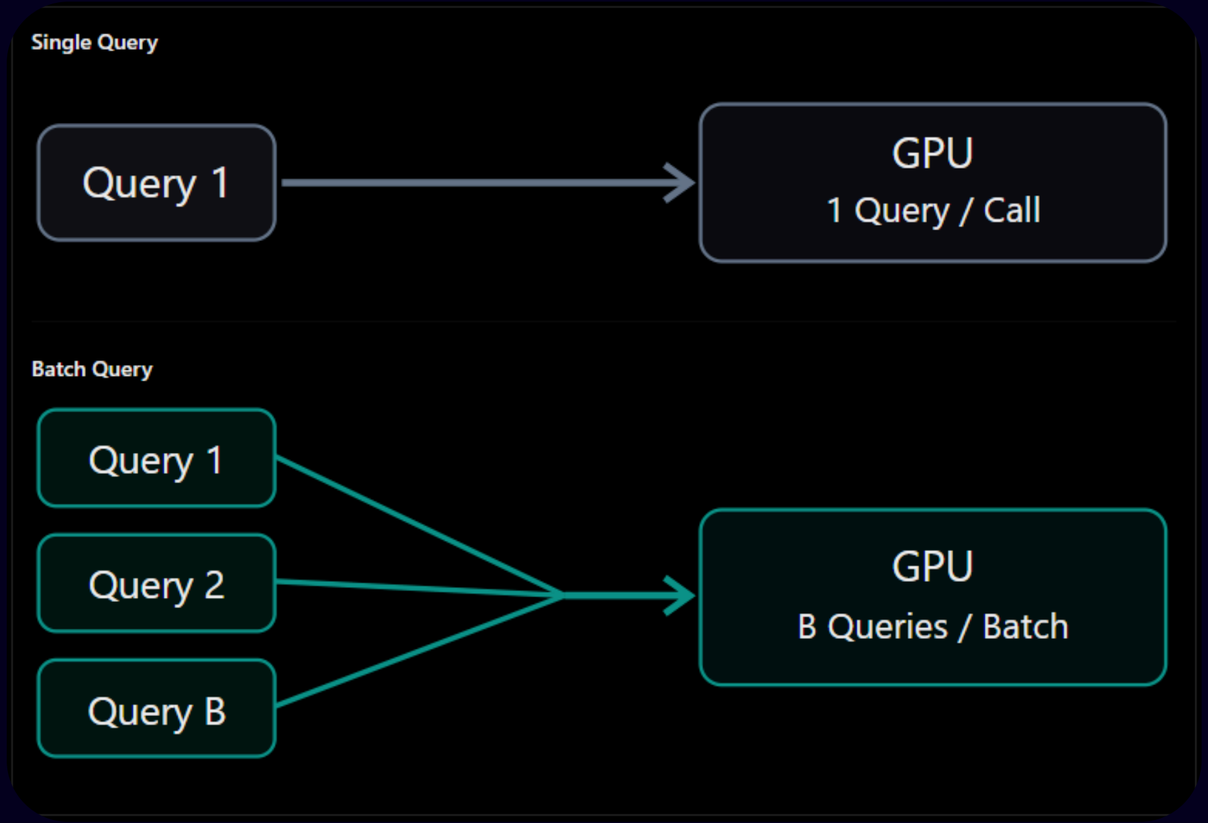
Key Finding

- V4 improved host-side transpose performance in some comparisons, but GPU Raw E2E did not outperform CPU in the tested comparisons.

Next Optimization

- V5 — Batch Query: Process multiple queries together to increase useful GPU work and amortize fixed overhead.

From Single Query to Batch Query



V5 increases useful GPU parallel work and amortizes overhead by processing multiple queries together, while preserving Exact Nearest Neighbor Search.

- Process multiple queries together
- Amortize overhead across queries
- Increased GPU workload parallelism
- Opportunity to improve GPU throughput

V5 — Batch Query Optimization & Experimental Setup

Dataset Configuration

Parameter	Configuration
Dataset	INRIA TexMex SIFT1M
Vector Dimension (D)	128
Candidate Count (N)	1,024 / 16,384 / 262,144 / 1,000,000
Batch Size (B)	8 / 32 / 128
Total Workloads	**12 (4 × 3)**
Search Type	Exact Nearest Neighbor
Primary Metric	GPU Raw E2E vs CPU Batch Raw E2E

Optimization Strategy

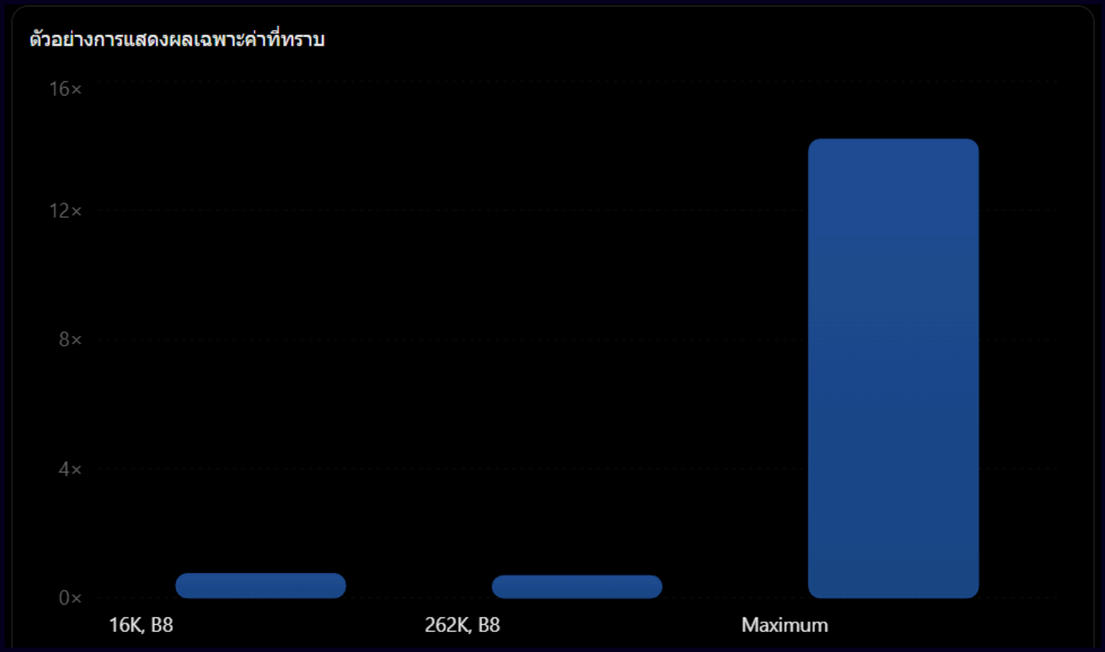
- Batch Processing: Process multiple queries together on GPU.
- Data Reuse: Reuse dataset and workspace across queries.
- Parallelism: Increase GPU utilization through larger workloads.
- Exact Search: Preserve exhaustive search without approximation.

Key Takeaway

V5 aims to amortize GPU overhead and improve performance through batch-level parallelism while maintaining Exact NN correctness.

V5 — GPU vs CPU Performance Results

V5 Speedup Across 12 Workloads



Performance Summary

10/12	≈14.2×
GPU Wins	**Maximum Speedup**
83.3% of workloads	GPU vs CPU Batch

Workloads Where GPU Is Slower

Candidates (N)	Batch Size (B)	Speedup	Result
16,384	8	0.74×	GPU Slower
262,144	8	0.68×	GPU Slower

GPU outperformed CPU in 10/12 SIFT1M batch workloads, reaching approximately 14.2× maximum speedup. Two B=8 workloads remained slower than CPU.

Conclusion & Future Work

1.Optimization Evolution

Version	Optimization
V0–V3	Kernel, Memory Layout & Reduction
V4	Cache-Blocked Transpose
V5	Batch Query Optimization

2.Key Results

- Dataset: INRIA TexMex SIFT1M (D = 128)
- GPU Wins: 10/12 Workloads (83.3%)
- Maximum Speedup: ≈14.2×
- Exact NN Correctness: PASS

3. Remaining Limitations

- N = 16,384, B = 8: GPU Speedup ≈0.74×
- N = 262,144, B = 8: GPU Speedup ≈0.68×

4. Future Work

- Optimize GPU performance for small batch sizes (B = 8).
- Reduce remaining GPU execution and memory overhead.
- Re-evaluate all 12 workloads under the same predefined acceptance criteria.

5. Final Takeaway

- GPU Batch Query outperformed CPU in 10/12 workloads, achieving up to 14.2× speedup, but the formal 12/12 performance criterion remains unmet.

The End